CANDIDATE PRACTICAL ASSIGNMENT

CloudVault

Secure Multi-Tenant Document Processing Platform

Target level: AWS Cloud Engineer • approximately 2 years of experience

Expected effort: 12–16 hours

Submission window: 4–5 calendar days

Primary focus: Architecture, security, automation, reliability and operations

Version: 1.0 • Candidate brief

Purpose: Design and deploy a production-minded AWS workload. Application code may be intentionally small; the assessment emphasizes sound cloud engineering decisions and evidence that the system works.

Independent hiring exercise • Not connected to any existing company project

1. Assignment overview

Build CloudVault, a secure AWS platform that accepts JSON or PDF documents, processes them asynchronously, tracks every job, isolates tenant data, and exposes enough operational evidence to investigate failures. The system must be deployed from code and must not depend on manually created production resources.

Scope boundary: A polished frontend is not required. A minimal API and simple processor are sufficient. Spend your time on AWS design, security, automation, monitoring, recoverability and clear engineering reasoning.

1.1 Engagement parameters

Item	Requirement
Expected effort	12–16 hours; document incomplete items instead of exceeding the time box.
Submission window	4–5 calendar days from receipt.
AWS account	Use a personal sandbox or an account supplied by the hiring team.
Region	Deploy to one AWS Region. Multi-region implementation is not required.
Infrastructure	Terraform, CloudFormation, AWS CDK or SAM. Console-only builds are not accepted.
Application language	Python, Node.js, Java, Go or another reasonable language.
Evidence	Provide deployment output, test evidence and screenshots or exported metrics as appropriate.

1.2 Success outcome

- A valid tenant document can be uploaded without making an S3 bucket public.
- The document is processed asynchronously and reaches a terminal status.
- Invalid documents are quarantined without being confused with technical failures.
- Temporary failures are retried and unrecoverable messages reach a dead-letter queue.
- Duplicate events do not create duplicate business outcomes.
- An EC2 failure is detected and the service recovers through Auto Scaling.
- Logs, metrics and audit events allow an operator to explain what happened to a job.
- All resources can be recreated and removed using documented commands.

2. Business scenario

A software company receives business documents from multiple customers. Each customer is represented as a tenant. Uploaded content may contain sensitive information, so storage must be private and encrypted. Processing demand is uneven: the platform may receive only a few files during normal operation and a sudden burst during a customer import.

The operations team needs an API for upload initiation and status lookup, automatic processing, alerts for failure, a daily summary, and evidence suitable for troubleshooting or a security review. Tenant A must never be able to read Tenant B’s job or document.

2.1 Mandatory AWS services

Capability	Required AWS services
Network and compute	Amazon VPC, EC2, EC2 Auto Scaling, Application Load Balancer, Systems Manager
Storage and processing	Amazon S3, AWS Lambda, Amazon SQS and an SQS dead-letter queue
State and scheduling	Amazon DynamoDB, Amazon EventBridge
Security	AWS IAM, AWS KMS, AWS Secrets Manager, AWS WAF
Operations and audit	Amazon CloudWatch, AWS CloudTrail, Amazon SNS
Delivery	Infrastructure as Code and a CI/CD pipeline

3. Reference architecture and workflow

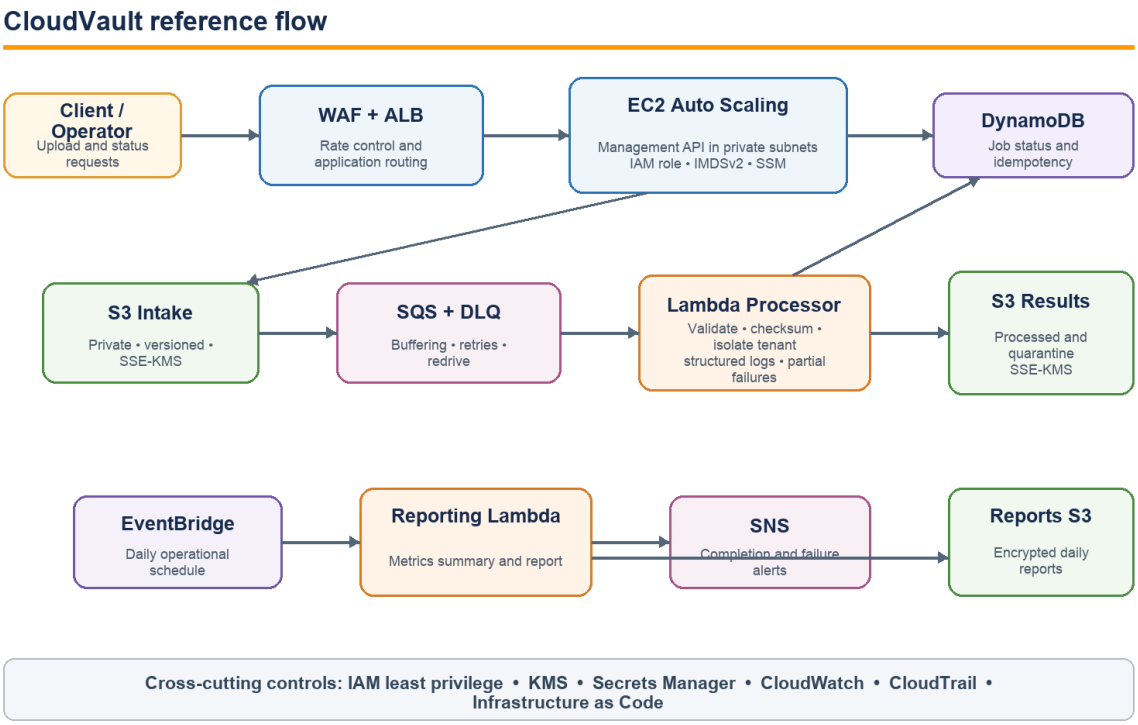

Figure 1. Reference flow. Equivalent architectures are acceptable when trade-offs are explained.

You may refine this architecture. Any change should preserve the required services and business outcomes and should be justified in the README.

3.1 End-to-end processing sequence

1. A client calls the EC2-hosted management API to create a job and request a pre-signed S3 upload URL.
2. The API creates a PENDING job in DynamoDB and returns a time-limited URL tied to the expected tenant and object key.
3. The client uploads a JSON or PDF document directly to the private S3 intake location.
4. S3 delivers an object-created event to the processing SQS queue.
5. Lambda consumes the message, validates the object and metadata, computes a SHA-256 checksum, and performs an idempotency check.
6. Valid documents are written to processed storage; invalid business documents are written to quarantine storage with a safe rejection reason.
7. DynamoDB reaches COMPLETED, REJECTED or FAILED, and SNS publishes the appropriate operational notification.

8. EventBridge invokes a reporting Lambda on a schedule and stores an encrypted operational report in S3.

4. Functional requirements

4.1 EC2 management API

Method and path	Required behavior
GET /health	Return process health and dependency-readiness information suitable for ALB health checks.
POST /api/v1/documents/upload-url	Validate request metadata, create a PENDING job and return a short-lived pre-signed upload URL.
GET /api/v1/documents/{jobId}	Return the job only when it belongs to the authenticated or asserted tenant.
GET /api/v1/documents	Return a tenant-scoped, paginated list without using a DynamoDB table scan.
GET /api/v1/operations/summary	Return basic counts for completed, rejected and failed jobs.

- Use versioned routes under /api/v1.
- Return consistent JSON error responses and appropriate HTTP status codes.
- Do not return internal stack traces, AWS credentials, raw secrets or another tenant's identifiers.
- A production authentication system is not required. Use a signed JWT, an API-key-to-tenant mapping, or a clearly documented test identity mechanism and explain the production replacement.

4.2 Accepted document and metadata rules

- Accept JSON and PDF only; verify extension and content type and document any stronger content validation used.
- Default maximum object size: 10 MB. Make the limit configurable.
- Require jobId, tenantId, customerId, documentType, fileName and createdAt metadata.
- Reject malformed JSON metadata and invalid identifiers without logging the document body.
- Calculate and persist a SHA-256 checksum for accepted content.
- Use deterministic output keys so replay does not create duplicate processed objects.

4.3 Job state model

```
PENDING -> PROCESSING -> COMPLETED
                      |-> REJECTED   (business validation failure)
                      |-> FAILED     (terminal technical failure)

Allowed terminal states: COMPLETED, REJECTED, FAILED
```

Use conditional updates or an equivalent concurrency control so that late, duplicate or replayed events cannot incorrectly move a terminal job backward.

5. Platform requirements

5.1 VPC, load balancing and EC2

- Create a custom VPC spanning at least two Availability Zones.
- Place the ALB in public subnets and EC2 application instances in private subnets.
- Use a launch template and Auto Scaling group. Production design must support at least two instances; a one-instance development setting is acceptable to control assessment cost.
- Configure an ALB health check and one target-tracking scaling policy.
- Do not assign public IP addresses to EC2 and do not expose SSH. Use Systems Manager Session Manager for administration.
- Require IMDSv2 and use an EC2 instance profile instead of access keys.
- Allow the application port only from the ALB security group.
- Run the API as a non-root service that starts on boot and restarts after a crash.
- Explain the choice between NAT Gateway, VPC endpoints or a combined design, including cost implications.

TLS during assessment: If you do not own a domain, HTTP is acceptable for a short-lived sandbox demonstration. Your production design must show ACM-managed TLS termination, secure redirection and an appropriate DNS approach.

5.2 S3 and document lifecycle

- Provide logical storage for intake, processed, quarantine and audit/report data. Separate buckets or clearly isolated prefixes are acceptable when justified.
- Enable Block Public Access, versioning and customer-managed KMS encryption.
- Deny non-TLS requests through bucket policy and restrict workload roles to required prefixes and operations.

- Use tenant-scoped object keys and avoid predictable access to another tenant’s content.
- Configure lifecycle rules and explain retention, deletion and recovery behavior.
- Demonstrate that an unauthenticated or unauthorized public object request is denied.

5.3 AWS KMS

- Create customer-managed symmetric keys for application documents and audit logs, with separate aliases.
- Enable automatic rotation and separate key administration from workload use.
- Grant decrypt or data-key permissions only to the roles and services that require them.
- Avoid broad principals and explain every wildcard retained in a key policy.
- Document the operational response to a disabled key and the safeguards around scheduled deletion.

5.4 SQS and Lambda processing

- Use an encrypted main queue and encrypted dead-letter queue with a documented redrive policy.
- Set the visibility timeout in relation to the Lambda timeout and explain the chosen values.
- Consume messages in batches and implement partial batch failure reporting.
- Use environment variables for resource identifiers and a dedicated Lambda execution role.
- Configure deliberate memory, timeout, batching and concurrency values.
- Use structured logs containing jobId, tenantId, outcome and duration; never log the document body.
- Separate a business rejection from a retryable technical error.
- Describe and demonstrate a safe DLQ replay procedure.

5.5 DynamoDB

- Model tenant-scoped status and list queries without relying on Scan for normal API operations.
- Store job state, source and result keys, checksum, timestamps, failure reason and attempt count.
- Use conditional writes or transactions for idempotency and safe state transitions.
- Enable point-in-time recovery and customer-managed KMS encryption.

- Use TTL only where the business retention decision supports deletion.
- Document partition keys, sort keys, secondary indexes and expected access patterns.

5.6 EventBridge, reporting and SNS

- Use EventBridge to invoke a daily reporting Lambda. A five-minute schedule may be used during demonstration.
- Report received, completed, rejected and failed counts; DLQ depth; and average processing duration.
- Store the report in encrypted S3 with a predictable date-based key.
- Publish SNS notifications for completion, rejection and significant operational failures.
- An email subscription may remain unconfirmed if you prefer not to provide a personal address.

5.7 Secrets Manager and configuration

- Store at least one realistic secret such as an application signing secret or external webhook token in Secrets Manager.
- Do not place the secret value in source code, EC2 user data, CI logs or committed variable files.
- Retrieve the secret through a workload IAM role and describe rotation and cache behavior.
- Keep non-secret configuration in environment variables, Parameter Store or another justified source.

5.8 WAF, IAM and tenant isolation

- Attach a WAF web ACL to the ALB with an AWS-managed common rule group, a rate-based rule and a request-size restriction.
- Use separate IAM roles for EC2, document processing, reporting and delivery automation.
- Do not use workload IAM users, long-lived access keys or AdministratorAccess.
- Avoid Action: * and Resource: * where resource-level permissions are available; justify unavoidable exceptions.
- Enforce tenant scope in API logic, DynamoDB access patterns and S3 object keys.
- Explain how WAF false positives and IAM authorization failures would be investigated.

6. Observability, audit and recovery

6.1 CloudWatch requirements

Area	Minimum telemetry
ALB	Request count, target response time, 4xx/5xx and healthy target count
EC2	CPU, status checks and application logs
Lambda	Invocations, duration, errors and throttles
SQS	Visible messages, age of oldest message and DLQ depth
Application	At least one custom metric for successful or rejected processing

- Create a single CloudWatch dashboard covering the minimum telemetry.
- Configure alarms for no healthy targets, EC2 status failure, Lambda errors or throttles, queue backlog, DLQ messages and elevated ALB 5xx responses.
- Set explicit retention on every application-created log group.
- Make one job traceable across EC2, S3 event processing and Lambda by jobId.

6.2 CloudTrail and audit evidence

- Create a trail for management events, write it to a dedicated encrypted S3 bucket and enable log-file validation.
- Restrict CloudTrail bucket and KMS access to intended principals and services.
- Enable relevant S3 data events or explain the cost decision and production recommendation.
- Document how you would investigate a deleted object, disabled KMS key, changed security-group rule and changed IAM policy.

6.3 Backup and recovery

- Use S3 versioning and DynamoDB point-in-time recovery.
- Define an RPO and RTO appropriate for this sandbox workload and a more realistic production target.
- Explain how infrastructure, configuration and data would be restored after accidental deletion.
- Describe—not implement—a regional disaster-recovery approach and identify which data requires replication.

7. Infrastructure as Code and CI/CD

7.1 Infrastructure as Code

- Create all important resources from code. Small verification actions in the console are acceptable; manual infrastructure dependencies are not.
- Use reusable modules, constructs or nested stacks for networking, compute, storage, IAM/KMS, processing, monitoring and audit concerns.
- Separate environment-specific inputs, validate important variables and use consistent resource tags.
- Return useful outputs such as the ALB endpoint, bucket names, queue URLs and dashboard name.
- Do not commit Terraform state, generated deployment credentials, secrets or .env files.
- If using Terraform, explain how remote state, locking, encryption and state access would be secured.

7.2 CI/CD

- Provide a pipeline for source checkout, formatting, IaC validation, static security checks, application tests, packaging and deployment to a development environment.
- Prefer OIDC or another short-lived credential mechanism for CI access to AWS.
- Do not store static AWS access keys in repository secrets unless you explicitly document why and how they would be replaced.
- A production approval stage is optional; describe the controls you would add for production.

8. Required verification scenarios

Provide an automated test, command transcript, screenshot or concise observation for each scenario. Clearly distinguish tests you executed from production recommendations you only documented.

#	Scenario	Expected evidence
1	Valid JSON upload	Job reaches COMPLETED and encrypted result exists.
2	Valid PDF upload	Supported content completes without being logged.
3	Unsupported extension	Job reaches REJECTED and object is quarantined.
4	Oversized object	Configurable limit is enforced safely.
5	Malformed metadata	Request or processing is rejected with a safe reason.
6	Missing tenant ID	No unscoped job or object is created.

#	Scenario	Expected evidence
7	Duplicate event	Only one business result is produced.
8	Temporary Lambda failure	Message is retried without corrupting job state.
9	Repeated technical failure	Message reaches DLQ and alarm evidence is visible.
10	Cross-tenant read	API denies access and does not reveal object metadata.
11	EC2 termination	Auto Scaling launches a replacement and health recovers.
12	Public S3 request	Anonymous or unauthorized access is denied.
13	API request burst	WAF rate rule or documented test threshold responds.
14	Cleanup	Destroy process removes assessment resources as documented.

9. Deliverables and repository structure

```
cloudvault/
├── infrastructure/
├── ec2-application/
├── lambda/
│   ├── document-processor/
│   └── daily-reporter/
├── tests/
├── diagrams/
├── scripts/
├── .github/workflows/ or pipeline/
├── README.md
├── SECURITY.md
├── RUNBOOK.md
└── COST-ESTIMATE.md
```

9.1 README expectations

- Architecture diagram and end-to-end data flow.
- Prerequisites and exact deploy, verify and destroy commands.
- IAM, KMS, network and tenant-isolation decisions.
- DynamoDB access patterns and idempotency strategy.
- Failure handling, DLQ replay and operational runbook links.
- Estimated monthly production cost and actual assessment cost where available.
- Assumptions, known limitations, incomplete items and next production improvements.

9.2 Submission package

- A private repository or archive containing source code and documentation.
- A short evidence folder or document showing successful and failure-path verification.

- A concise presentation or README section suitable for a 20-minute technical walkthrough.
- No credentials, secret values, private keys, state files or customer information.

10. Demonstration acceptance checklist

1. Deploy the infrastructure using the documented process.
2. Show EC2 targets registered behind the ALB and explain security-group boundaries.
3. Create an upload job and upload a valid document using the pre-signed URL.
4. Show the job status, encrypted processed object and matching DynamoDB record.
5. Show invalid-document quarantine and a safe rejection reason.
6. Show duplicate-event behavior, Lambda retry behavior and a message reaching the DLQ.
7. Locate one job in structured logs and show the relevant dashboard or alarm.
8. Find an infrastructure change or object action in CloudTrail.
9. Terminate an EC2 instance and show Auto Scaling replacement behavior.
10. Demonstrate that public S3 access and cross-tenant status access are denied.
11. Run or explain the complete infrastructure cleanup process.

11. Evaluation rubric

Evaluation area	Points	What strong evidence looks like
Architecture and service selection	10	Coherent data flow, sound trade-offs and limited unnecessary complexity.
VPC, ALB, EC2 and Auto Scaling	15	Private compute, secure access, health checks, bootstrapping and recovery.
Lambda, SQS and event processing	15	Retries, DLQ, partial failure, idempotency and correct error classification.
IAM, KMS, S3 and tenant security	20	Least privilege, private encryption, safe key policy and tenant isolation.
Infrastructure as Code	15	Reproducible, modular, validated and cleanly destroyable infrastructure.
Monitoring, audit and recovery	10	Useful telemetry, alarms, CloudTrail evidence and recovery reasoning.
CI/CD and automation	5	Safe short-lived authentication and meaningful validation stages.
Testing and failure handling	5	Credible success, rejection, retry, replay and isolation tests.

Evaluation area	Points	What strong evidence looks like
Documentation and communication	5	Clear instructions, decisions, limitations and technical explanation.

Recommended pass threshold: 65/100 overall, including at least 12/20 in IAM, KMS, S3 and tenant security. A strong explanation of an intentional trade-off may receive credit even when the implementation is simplified for time or cost.

11.1 Critical security failures

A submission may be rejected regardless of total score when it includes any of the following:

- Committed AWS credentials, private keys or real secret values.
- Public access to document buckets or deliberate exposure of tenant content.
- AdministratorAccess attached to an application workload.
- SSH exposed to 0.0.0.0/0 or public EC2 access without a defensible constraint.
- Unencrypted document storage or hardcoded credentials on EC2 or Lambda.
- No meaningful tenant isolation, retry handling or Infrastructure as Code.
- A deployment that cannot be reproduced from the submitted documentation.

12. Cost, safety and professional conduct

- Use small development-sized resources and deploy the full stack only long enough to gather evidence.
- Create a low AWS Budget alert where account permissions allow it.
- Destroy NAT Gateways, load balancers, WAF resources and EC2 instances promptly after the demonstration.
- Do not use real customer data, production credentials or an employer-owned account without authorization.
- Never publish sensitive repository content to a public location.
- If a required service is unavailable in the account, document the limitation and provide deployable code rather than silently omitting it.

Cost expectation: The assessment is designed for a short-lived sandbox deployment. Estimate cost before deployment and aim to keep actual spend low. Free-tier eligibility is not assumed.

Appendix A. Reference data contract

```
{
  "jobId": "550e8400-e29b-41d4-a716-446655440000",
  "tenantId": "tenant-001",
  "customerId": "CUST-123",
  "documentType": "invoice",
  "fileName": "invoice-2026-001.pdf",
  "amount": 1250.50,
  "createdAt": "2026-07-22T10:00:00Z"
}
```

Appendix B. Suggested DynamoDB attributes

Attribute	Purpose
tenantId	Tenant partition boundary or part of the selected composite key.
jobId	Immutable job identifier.
status	PENDING, PROCESSING, COMPLETED, REJECTED or FAILED.
sourceObjectKey	Tenant-scoped S3 intake key.
processedObjectKey	Processed or quarantine key when present.
checksum	SHA-256 checksum for accepted content.
createdAt / updatedAt	UTC timestamps used for ordering and operational evidence.
failureReason	Sanitized reason suitable for operators; no raw document body.
attemptCount	Processing attempts or a justified equivalent.

Appendix C. Final submission checklist

- All mandatory services are represented in code or an explicitly documented limitation.
- Deployment, verification and cleanup commands have been run from a clean environment where practical.
- No credentials, secrets, state files, build artifacts or sensitive values are committed.
- The architecture diagram matches the deployed solution.
- Successful, rejected, retry and DLQ paths have evidence.
- Security assumptions, costs, production gaps, RPO and RTO are documented.
- The repository is accessible to the hiring team for the agreed review period.